

Question 1: Heuristic design and admissibility [15 points]

For each of the three cost measures—steps, Scrabble, and frequency—describe the heuristic implemented in your Program.

For each heuristic:

1. Give a precise formula or pseudocode.
2. Explain what unavoidable remaining costs of lower-bounds.
3. Explain why it cannot overestimate the true optimal remaining cost.

Your explanation must be specific to the corresponding cost measure. Merely stating that the heuristic “is a lower bound” is insufficient.

STEPS

```
Python
diff = 0

for each letter i in the word:
    if state[i] != goal[i]:
        diff = diff + 1
H(state) = diff
```

The unavoidable cost of lower bounds is that every letter that doesn't match the goal has to change at some point and there's no way to avoid it. Each move changes only one letter and in steps every move costs exactly 1. So if you have cot as a state and cat as a goal you would at least have to change one letter to be able to reach the goal state.

This does not overestimate because one move can fix at most one wrong letter. If letters are wrong, it takes at least diff moves to fix them, and each move costs exactly 1, so the real cost is at least diff. The real cost can be higher if a letter can't be changed directly because the word in between isn't in the dictionary, which forces extra detour moves but it can never be lower than diff.

SCRABBLE

```
Python
diff = 0
```

```

for each letter i in the word:
    if state[i] != goal[i]:
        diff = diff + 1
H(state) = diff * 1

```

For Scrabble, each move costs the Scrabble value of the new letter you put in. Every letter that does not match the goal has to be changed at least once and the cheapest letters cost 1. So you count all the mismatched letters and multiply them by the minimum cost of 1 and you get the smallest the Scrabble cost can be.

This does not overestimate because every wrong letter needs at least one move, and every move costs at least 1, so the real cost is at least diff times 1. The real cost is often higher because the letter that you may need might be worth more than 1 and detours can add extra moves.

FREQUENCY

```

Python
diff = 0

for each letter i in the word:
    if state[i] != goal[i]:
        diff = diff + 1
H(state) = diff * 1

```

For frequency each move costs 1 plus the number that is stored for the new word in the dictionary. The dictionary numbers are never negative and the smallest they can be is 0. That means every move costs at least 1.

For cost you need at least one move for each wrong letter because each move changes one letter and every move costs at least one because the dictionary number can't be negative. This means the cost will be at least the wrong amount of letters times 1.

Question 2: Does the heuristic actually help? [10 points]

Run both uniform-cost search and A* on:

- WHY → ASK
- BEAR → DUCK

Calculate the percentage reduction as:

$$100 \times \frac{\text{UCS expansions} - \text{A* expansions}}{\text{UCS expansions}}$$

A negative result means that A* expanded more nodes than UCS.

Word pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY → ASK	Steps	5	773	5	16	97.9%
WHY → ASK	Scrabble	9	765	9	353	53.9%
WHY → ASK	Frequency	38.8554	808	38.8554	755	6.6%
BEAR → DUCK	Steps	4	691	4	6	99.1%
BEAR → DUCK	Scrabble	11	2981	11	1376	53.8%
BEAR → DUCK	Frequency	38.240	1084	38.240	720	33.6%

Then answer:

1. Did UCS and A* always return the same optimal cost? If not, identify and investigate the discrepancy.

Yes UCS and A* always returned the same cost

2. For each word pair, under which cost measure did A* produce the largest percentage reduction in expanded Nodes?

Steps produced the largest reduction in expanded nodes because BEAR → DUCK and WHY → ASK had 99.1% reduction and 97.9% reduction respectively which is much higher than the other word pairs and their cost measure.

3. For each word pair, under which cost measure was the heuristic least informative?

The cost measure that was least informative was Frequency with WHY → ASK being improved by 6.6% and BEAR → DUCK was improved by 33.6% which are the smallest reductions on the table

4. Explain the differences using the definitions of your heuristics, not runtime alone.

All my heuristics use the same formula which is counting the wrong letters and multiplying by 1. The thing that changes is how close that guess is to the real cost for each cost measure. When the guess is closer to the real cost it's easier for A* to expand less nodes because it can tell which paths are worth exploring and which paths are not.

For steps every move costs exactly 1 so the number of wrong letters is really close to the real cost. The only thing it misses is detour moves so the guess is almost always right or very close. That's why steps had the biggest reduction for both word pairs.

For Scrabble the heuristic still counts 1 for each wrong letter but the real moves can cost up to 10 depending on the letter you put in. Since the heuristic doesn't count how much the letters are actually worth the guess ends up further from the real cost. That means A* can't skip as many paths so the reduction goes down.

For frequency each move costs 1 plus the rarity of the new word but the heuristic doesn't count the rarity at all. The rarity numbers are pretty big so most of the real cost comes from rarity and the heuristic misses all of it. The guess is way too low to help A* much so A* ends up acting almost the same as UCS and that's why frequency had the smallest reduction.

Different optimal paths are acceptable when their total costs are equal.

Question 3: Reversibility and direction-dependent costs [5 points]

DOGCAT actions are reversible: if one legal word can be changed into another, the reverse change is also legal.

For each cost measure—steps, Scrabble, and frequency—determine whether the optimal cost from W1 to W2 must always equal the optimal cost from W2 to W1.

For each cost measure:

1. State yes or no.
2. Justify your conclusion.
3. If the answer is no, give a concrete counterexample using words from the provided dictionary.

Clearly distinguish between the reversibility of an action and the symmetry of its cost.

All DOGCAT actions are reversible which means if you can change W1 into W2 you're also allowed to change W2 back into W1 because both words are in the dictionary. This is true for all three cost measures. Cost symmetry is different because it means the move costs the same going both ways. Reversibility only means you're allowed to go back but it doesn't mean going back costs the same.

STEPS

1. Yes
2. Every move costs exactly 1 no matter which letter you put it in or which word you land on. If the cheapest path from W1 to W2 takes 5 moves it should also take 5 steps to get back to W2 from W1. Both directions can't be cheaper than another

FREQUENCY

1. No
2. Each move costs 1 plus the number of the word you are going in. Going forward you pay for the word ahead but going backward you pay for the word before which have different costs
3. $W1 \rightarrow W2$ costs 1 plus the cost of $W2$; $W2 \rightarrow W1$ costs 1 plus the cost of $W1$
 - a. $CAT \rightarrow HAT = 1 + 6.45 = 7.45$; $HAT \rightarrow CAT = 1 + 6.89 = 7.89$
 - b. $7.45 \neq 7.89$

SCRABBLE

1. No
2. The cost is based on the cost of the letter you put in and both ways to and from will be with different because it won't be the same numbers. One way can be cheaper than going back the other way
3. $CAT \rightarrow HAT$ replace c with h = cost is 4; $HAT \rightarrow CAT$ replace h with c = cost is 3

Question 4: LLM heuristic audit [10 points]

Give a generative AI system the following prompt:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic $b_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than b_0 while remaining admissible. Define it precisely and justify your proposal.

By “more informative,” we mean that the proposed heuristic should be at least as large as b_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Submit:

1. The model used, the exact prompt, and its proposed heuristic. A complete conversation transcript is not required.
2. A precise formula or pseudocode for the proposed heuristic.
3. Determine whether the heuristic is admissible. Give either a general proof or a concrete counterexample.
4. If the proposal is invalid, make the smallest repair you can or replace it with a safe alternative.
5. Test the verified or repaired heuristic against b_0 on the two frequency-cost problems from Question 2. Compare solution costs and numbers of expanded nodes, then conclude whether the evidence supports using the proposed heuristic.

Experimental tests can reveal counterexamples, but passing a collection of tests does not by itself prove admissibility. The AI system is not expected to be correct. Your grade is based on the quality of your verification, correction, and Justification.

1. Model used is Opus 5.5 medium

None

The prompt:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal. By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining

Show more

Admissibility

Take any state $n \neq g$ and any path from n to g with k actions.

1. **Number of steps.** Each action changes exactly one letter, so it reduces the number of mismatched positions by at most 1. Therefore $k \geq h_0(n) \geq 1$.
2. **The last step.** The final action enters g , so it costs exactly $1 + \text{rarity}(g)$.
3. **The other steps.** Each of the remaining $k - 1$ actions enters some word in D_L , so each costs at least $1 + r_{\min}$.

So the path cost is at least $(1 + \text{rarity}(g)) + (k - 1)(1 + r_{\min})$. Since $1 + r_{\min} > 0$ and $k \geq h_0(n)$, this is at least $h_R(n)$. This holds for every path, including the optimal one, so $h_R(n) \leq h^*(n)$. At the goal, $h_R(g) = 0 = h^*(g)$.

Consistency (a bonus)

Consider an action $n \rightarrow n'$ with cost $c = 1 + \text{rarity}(n')$.

- If $n' = g$: then $h_0(n) = 1$, so $h_R(n) = 1 + \text{rarity}(g) = c + h_R(g)$.
- If $n' \neq g$: h_0 changes by at most 1, so $h_R(n) - h_R(n') \leq 1 + r_{\min} \leq c$.

Thus $h_R(n) \leq c(n, n') + h_R(n')$. A* with graph search therefore never needs to reopen nodes.

More informative than h_0

At least as large everywhere. At g , both heuristics are 0. For $n \neq g$, since rarities are nonnegative:



Optional refinement (still admissible)

When $h_0(n) \geq 2$, the second-to-last word on any path must differ from g in exactly one position. Let

$$r_1 = \min\{\text{rarity}(w) : w \in D_L, h_0(w) = 1\}$$

Then for $h_0(n) \geq 2$:

$$h_R'(n) = (1 + \text{rarity}(g)) + (1 + r_1) + (h_0(n) - 2)(1 + r_{\min})$$

Since $r_1 \geq r_{\min}$, this satisfies $h_R' \geq h_R$ and is admissible by the same argument. If no dictionary word is one letter from g , every state other than g is unreachable, so the goal can only be reached from itself.

Tiny example

Suppose the goal is **cat** with rarity 0.5, $r_{\min} = 0.1$, and the state is **dog**, so $h_0 = 3$.

- $h_0(\text{dog}) = 3$
- $h_R(\text{dog}) = 1.5 + 2(1.1) = 3.7$

The true optimal cost must be at least 3.7, so h_R is admissible and strictly more informative here.

📄 🔍 ⬅️ ➡️ ⌂ ⌚ 7 hours ago



Deriving cost bounds for each step toward the goal word. ▾

Thought for 15s

Designing a heuristic for an AI search problem.

Deriving cost bounds for each step toward the goal word.

A Rarity-Aware Admissible Heuristic for DOGCAT

Setup and notation

Let g be the goal word and L its length. Let D_L be the set of dictionary words of length L . Every legal state lies in D_L . Define

$$r_{\min} = \min\{\text{rarity}(w) : w \in D_L\}$$

This is computed once per problem in one pass over the dictionary. Entering any word costs at least $1 + r_{\min}$.

Proposed heuristic

$$h_R(n) = 0, \text{ if } n = g$$

$$h_R(n) = (1 + \text{rarity}(g)) + (h_0(n) - 1)(1 + r_{\min}), \text{ otherwise}$$

In words: any path to the goal must end by entering g , and that step's cost is known exactly. Every earlier step costs at least the cheapest possible entry cost.

$$h_R(n) \geq 1 + (h_0(n) - 1) = h_0(n)$$

Strictly larger somewhere. The difference is

$$h_R(n) - h_0(n) = \text{rarity}(g) + (h_0(n) - 1)r_{\min}$$

- If $\text{rarity}(g) > 0$, every non-goal state gets a strictly larger value, including all states one letter from the goal.
- If $r_{\min} > 0$, every state with $h_0(n) \geq 2$ gets a strictly larger value.

Caveat. If every relevant rarity is 0, the objective reduces to step counting. No rarity-based term can then add information, and h_R coincides with h_0 . It is never worse.

Why this design

h_0 counts steps but prices each step at 1, ignoring rarity entirely. h_R keeps the same step lower bound and prices those steps more accurately:

- The final step is priced exactly, since its destination is always g .
- The intermediate steps are priced at a guaranteed floor.

This is the tightest bound you can get without knowing which intermediate words the path uses.

Optional refinement (still admissible)

When $h_0(n) \geq 2$, the second-to-last word on any path must differ from g in exactly one position. Let

$$r_1 = \min\{\text{rarity}(w) : w \in D_L, h_0(w) = 1\}$$



Python

min = the smallest number in the dictionary amongst all words with the same length as goal

```
h_min(state):
    if state == goal:
        return 0
    diff = number of positions where state[i] does not equal goal[i]
    return (1 + dictionary[goal] + (diff - 1) * (1 + min))
```

2. Yes it is admissible because if you pick any word that isn't the goal and any path from it with x amount of moves there are enough moves to solve at most one letter so $x \geq \text{diff}$; the last move always lands on the goal word so it costs the goal dictionary cost + 1, and the other moves each land on some word and no word has a smaller cost

than the min so each move costs $1 + \min$. Therefore every path costs at least $1 + \text{dictionary goal cost} + (x - 1)(1 + \min)$. Since $x \geq \text{diff}$ every path costs at least $h_{\min}$ so it never overestimates. This only works if none of the rarity numbers are negative. I checked by printing the smallest number in the dictionary and it was 0.016226212 so none of them are negative. The heuristic works but the AI made a mistake when explaining it. In its tiny example it made up numbers and used them to say the heuristic was admissible but made up numbers can't prove that.

3. There is no repair needed :)

Word Pair	h0 Cost	h0 Nodes	h_min Cost	h_min Nodes	Reduction
WHY → ASK	38.855	755	38.855	634	16.0%
BEAR → DUCK	38.240	720	38.240	257	64.3%

With the table the evidence supports using the proposed heuristic because it reduced the amount of nodes expanded and still came up with the same optimal cost as h0. These tests don't prove admissibility by themselves because passing two tests can't rule out a counterexample somewhere else but the proof from before does.

AI DISCLAMER:

I also used Claude to help me while writing my answers. I used it to help me explain my heuristics in simpler words and to check my drafts for mistakes. It also helped me understand the reversibility and cost symmetry question, and write the DCR class that I used to test $h_{\min}$. All of the search results, node counts, and solution costs came from running my own code. I went over and edited everything the AI helped with, and I take responsibility for my final answers.